# Rekursion – Konzepte, Strukturen und Umsetzung in funktionalen Programmiersprachen

aus dem Studiengang Wirtschaftsinformatik

an der Duale Hochschule Baden-Württemberg Mannheim

von

Emanuel Schuessler

Jannes Borgmann

08.05.2026

**Matrikelnummer, Kurs:** ???, WI25SEA
???, WI25SEA

**Unternehmen:** SAP SE, 69190, Walldorf
SAP SE, 69190, Walldorf

**Betreuer an der DHBW:** Edelbert Schumm

# Inhaltsverzeichnis

**1 Einleitung** ..... **1**

**2 Definition und Grundlagen** ..... **2**
2.1 Was ist Rekursion? ..... 2
2.2 Historischer Hintergrund ..... 2
2.3 Strukturelle Rekursion ..... 3

**3 Rekursive Funktionen** ..... **4**
3.1 Das Fakultätsbeispiel ..... 4
3.2 Terminierung ..... 5
3.3 Endrekursion ..... 5
3.4 Weitere Varianten ..... 6

**4 Rekursive Datenstrukturen** ..... **7**

**5 Rekursion in der funktionalen Programmierung** ..... **8**

**6 Fazit** ..... **10**

**Literatur** ..... **a**

# Glossar

**Basisfall** Der Abbruchfall einer rekursiven Funktion, der ohne weiteren Selbstaufruf einen Wert zurückgibt und so die Terminierung der Rekursion sicherstellt.

**Endrekursion** Eine spezielle Form der Rekursion, bei der der rekursive Aufruf die letzte Operation der Funktion ist. Viele Compiler können Endrekursion in eine Schleife umwandeln (Tail Call Optimization).

**Pattern_Matching** Ein Sprachkonstrukt in funktionalen Programmiersprachen, das es erlaubt, Werte anhand ihrer Struktur zu unterscheiden und zu destrukturieren.

**Rekursion** Ein Konzept, bei dem sich eine Funktion oder Datenstruktur direkt oder indirekt auf sich selbst bezieht. Eine rekursive Definition besteht stets aus einem Basisfall (Abbruchbedingung) und einem allgemeinen Fall (Selbstaufruf).

**Selbstaufruf** Der Teil einer rekursiven Funktion, in dem die Funktion sich selbst mit einem vereinfachten Argument aufruft.

**Strukturelle_Rekursion** Eine Form der Rekursion, die der Struktur des Eingabedatentyps folgt, z.B. bei der Verarbeitung von Listen oder Bäumen.

# 1 Einleitung

Rekursion begegnet uns häufiger, als man zunächst denkt. Ob das Blätterwerk eines Baumes, verschachtelte Ordnerstrukturen auf dem Computer oder die Grammatik einer Programmiersprache – viele Dinge sind von Natur aus rekursiv aufgebaut. In der Mathematik kennt man das schon lange: Die Fakultät, Folgen oder der euklidische Algorithmus basieren alle auf demselben Prinzip, auch wenn es dort selten so genannt wird.

In der funktionalen Programmierung ist Rekursion noch zentraler: Da Sprachen wie Haskell keine veränderbaren Variablen kennen, gibt es auch keine klassischen Schleifen. Stattdessen werden Wiederholungen immer durch Rekursion ausgedrückt [1]. Diese Ausarbeitung erklärt, was Rekursion ist, wie sie funktioniert, welche Varianten es gibt und wie sie in der Praxis eingesetzt wird [2], [3].

# 2 Definition und Grundlagen

## 2.1 Was ist Rekursion?

Moritz Lenz beschreibt Rekursion treffend so: *„Eine Funktion macht nur einen kleinen Teil der Arbeit, verkleinert das Problem damit ein bisschen, und ruft sich dann selbst auf, um den Rest zu lösen.„* [3] Lorenz ergänzt: *„Man definiert etwas nicht explizit, sondern durch einfachere Versionen seiner selbst.“* [4]

Das Prinzip dahinter heißt **rekursive Problemreduktion**: Ein Problem wird auf ein strukturgleiches, aber kleineres Problem zurückgeführt – immer wieder, bis man bei einem Fall ankommt, den man direkt lösen kann [2]. Die wiederholte Ausführung überlässt man dabei komplett dem Programmiersystem, man beschreibt nur die Struktur der Lösung.

Damit das nicht ewig weitergeht, braucht jede Rekursion zwei Teile:

- **Basisfall** (auch Rekursionsanfang): Der einfachste Fall, der direkt beantwortet werden kann – ohne weiteren Selbstaufruf. Er sorgt dafür, dass die Rekursion irgendwann aufhört.
- **Allgemeiner Fall** (auch Reduktionsschritt): Das aktuelle Problem wird auf ein kleineres Problem desselben Typs zurückgeführt. Wichtig: Die Verkleinerung muss nach endlich vielen Schritten zum Basisfall führen [2].

## 2.2 Historischer Hintergrund

Den ältesten bekannten rekursiven Algorithmus hat Euklid im dritten Jahrhundert vor Christus beschrieben: das Verfahren der wechselnden Wegnahme zur Berechnung des größten gemeinsamen Teilers. Die Idee: Zwei Zahlen $a$ und $b$ haben denselben größten gemeinsamen Teiler wie $b$ und der Rest der Division $a$ durch $b$. Man wiederholt das so lange, bis der Rest Null ist – dann ist man fertig [4].

Im 19. Jahrhundert formalisierte Giuseppe Peano die natürlichen Zahlen durch seine Peano-Axiome. Darin sind die natürlichen Zahlen induktiv definiert: Die Null existiert, und jede

weitere natürliche Zahl ist der Nachfolger einer vorherigen. Grundoperationen wie Addition und Multiplikation lassen sich darauf aufbauend rekursiv definieren [4].

Im 20. Jahrhundert untersuchten Kurt Gödel und Alonzo Church, welche Funktionen überhaupt berechenbar sind. Gödel arbeitete dabei mit dem Konzept der primitiv-rekursiven Funktionen. In der Informatik wurde Rekursion dann mit der Sprache LISP (1958, John McCarthy) zum praktischen Werkzeug: McCarthy erkannte, dass man viele Berechnungen direkt als rekursive Funktionen schreiben kann.

## 2.3 Strukturelle Rekursion

Besonders wichtig ist die **strukturelle Rekursion**: Eine Funktion folgt dabei dem Aufbau des Datentyps, den sie verarbeitet. Wenn ein Datentyp selbst rekursiv definiert ist, ergibt sich die passende Funktion darüber fast automatisch [4].

Das zeigt sich gut an den natürlichen Zahlen: Man kann sie induktiv beschreiben – entweder ist eine Zahl die Null (Basisfall), oder sie ist der Nachfolger einer anderen natürlichen Zahl (allgemeiner Fall). Eine Funktion über natürliche Zahlen behandelt genau diese zwei Fälle:

| | **Natürliche Zahlen** | **Rekursive Funktion f(n)** |
|---|---|---|
| **Basisfall** | Die Zahl 0 existiert. | Für $n = 0$ gibt es eine direkte Antwort, z.B. $f(0) = 1$ bei der Fakultät. |
| **Allg. Fall** | Jedes $n > 0$ ist der Nachfolger von $n - 1$. | Für $n > 0$ wird $f(n-1)$ berechnet und mit $n$ verknüpft: $f(n) = n \cdot f(n-1)$. |

Tabelle 1 — Strukturelle Entsprechung zwischen Datentyp und Funktion

Das Prinzip: Wer den Aufbau des Datentyps kennt, weiß bereits, welche Fälle die Funktion behandeln muss.

# 3 Rekursive Funktionen

## 3.1 Das Fakultätsbeispiel

Die Fakultät ist das klassische Beispiel. Sie ist definiert als das Produkt aller ganzen Zahlen von 1 bis $n$, also z.B. $4! = 1 \cdot 2 \cdot 3 \cdot 4 = 24$. Rekursiv lässt sich das so ausdrücken: Die Fakultät von 0 ist 1 (Basisfall), und für alle anderen $n$ gilt $n! = n \cdot (n-1)!$ (allgemeiner Fall).

In Java schreibt sich das fast genauso:

```java
public static long factorial(int n) {
    if (n == 0) {
        return 1;                    // Basisfall
    }
    return n * factorial(n - 1); // allgemeiner Fall
}
```

Listing 1 — Rekursive Berechnung der Fakultät in Java

Was passiert beim Aufruf `factorial(4)`? Abbildung 2 zeigt den Aufrufstapel (Call Stack): Die Methode ruft sich selbst auf, bis sie bei `factorial(0)` ankommt und direkt 1 zurückgibt. Danach werden die Ergebnisse schrittweise rückwärts aufgelöst.

| Aufbau des Stapels |
|---|
| `factorial(4)` wartet auf `factorial(3)` |
| `factorial(3)` wartet auf `factorial(2)` |
| `factorial(2)` wartet auf `factorial(1)` |
| `factorial(1)` wartet auf `factorial(0)` |
| `factorial(0)` ⟶ gibt **1** zurück |

→

| Auflösung des Stapels |
|---|
| `factorial(1)` = 1 × 1 = **1** |
| `factorial(2)` = 2 × 1 = **2** |
| `factorial(3)` = 3 × 2 = **6** |
| `factorial(4)` = 4 × 6 = **24** |
| Ergebnis: **24** |

Tabelle 2 — Aufrufstapel bei factorial(4) – Aufbau (links) und Auflösung (rechts)

## 3.2 Terminierung

Eine Rekursion muss irgendwann aufhören. Das funktioniert nur, wenn das Argument bei jedem Aufruf strikt kleiner wird und irgendwann den Basisfall erreicht [2], [4]. Bei der Fakultät nimmt $n$ bei jedem Schritt um genau 1 ab – das führt zwingend zu $n = 0$.

Fehlt der Basisfall oder wird das Argument nicht verkleinert, läuft die Rekursion endlos. In Java endet das mit einem `StackOverflowError`, weil für jeden offenen Aufruf Speicher auf dem Aufrufstapel belegt wird – und der ist nicht unbegrenzt groß.

## 3.3 Endrekursion

Genau dieser Speicherverbrauch ist ein praktisches Problem bei tiefer Rekursion. Bei der normalen Fakultät muss Java sich für jeden offenen Aufruf merken, mit welchem $n$ es noch multiplizieren muss. Das ergibt bei $n = 10.000$ auch 10.000 offene Einträge auf dem Stack.

Die **Endrekursion** (englisch: tail recursion) löst das: Eine Funktion ist endrekursiv, wenn der rekursive Aufruf die allerletzte Operation ist und das Ergebnis direkt zurückgegeben wird – ohne dass danach noch etwas damit passiert. Der Compiler kann dann den alten Stackeintrag einfach wiederverwenden. Dafür führt man einen Akkumulator ein, der das Zwischenergebnis mitträgt:

Statt `return n * factorial(n-1)` (Multiplikation nach dem Aufruf) schreibt man den rekursiven Aufruf so, dass die Multiplikation bereits vorher erledigt und als Parameter übergeben wird.

Warum hilft das in Java nicht? Die JVM (Java Virtual Machine) unterstützt Endrekursionsoptimierung grundsätzlich nicht automatisch. Das liegt daran, dass die JVM vollständige Stack-Traces für Debugging und Fehleranalyse benötigt – sie muss also jeden Aufruf im Stapel behalten [5]. Scala löst das anders: Die Annotation `@tailrec` weist den Scala-Compiler an, eine endrekursive Funktion **vor** der Übersetzung in JVM-Bytecode in eine einfache Schleife umzuwandeln. Es handelt sich also um eine Compiler-Transformation, nicht um ein JVM-Feature. In Java bleibt man deshalb besser bei einer `for`-Schleife, wenn der Eingabewert groß sein kann [4].

## 3.4 Weitere Varianten

Bei der **gegenseitigen Rekursion** rufen sich zwei Funktionen abwechselnd auf. Zum Beispiel: `istGerade(n)` ruft `istUngerade(n-1)` auf, und umgekehrt. Beide bilden zusammen eine Rekursion, bis der Basisfall ($n = 0$) erreicht ist [3].

Bei der **verschachtelten Rekursion** ist der Selbstaufruf selbst wieder ein Argument eines weiteren Selbstaufrufs. Das bekannteste Beispiel ist die **Ackermann-Funktion**. Sie ist für alle natürlichen Zahlen berechenbar und terminiert immer – aber sie wächst so extrem schnell, dass sie schon für kleine Eingaben astronomische Werte annimmt.

Wichtig dabei: Die Ackermann-Funktion ist **nicht** primitiv-rekursiv. Primitiv-rekursiv bedeutet, dass sich eine Funktion auf eine Schleife mit vorher fest bekannter Schrittzahl zurückführen lässt – man weiß also bereits vor dem Start, wie oft man wiederholt. Einfache Funktionen wie Fakultät oder Addition sind primitiv-rekursiv. Bei der Ackermann-Funktion hingegen hängt die Anzahl der Schritte selbst von einem rekursiv berechneten Wert ab – sie entzieht sich damit jeder Schleife mit fester Laufzahl. Das zeigt, dass Rekursion ausdrucksstärker ist als Schleifen [4].

Ein praktisches Beispiel für Rekursion aus dem Alltag der Softwareentwicklung ist **Merge Sort**: Der Algorithmus teilt eine unsortierte Liste in der Mitte, sortiert beide Hälften jeweils rekursiv und fügt sie danach sortiert wieder zusammen. Der Basisfall ist eine Liste mit nur einem Element – die ist bereits sortiert. Lenz beschreibt das Prinzip so: Rekursion zerlegt größere Probleme auf natürliche Weise in kleinere, was die Lösung erheblich vereinfacht [3]. Merge Sort erreicht damit eine Laufzeit von $O(n \log n)$ – deutlich besser als einfache Verfahren wie Bubblesort mit $O(n^2)$. Er ist kein akademisches Konstrukt, sondern in vielen Standardbibliotheken als realer Sortieralgorithmus implementiert [6].

# 4 Rekursive Datenstrukturen

Nicht nur Funktionen, sondern auch Datenstrukturen können rekursiv aufgebaut sein. Man nennt sie **induktive Datentypen**: Ihre Definition besteht aus einem Basisfall und einem allgemeinen Fall, der auf den Typ selbst verweist [7].

**Listen** sind das bekannteste Beispiel. Eine Liste ist entweder leer (Basisfall) oder sie hat ein erstes Element (Kopf) und dahinter eine weitere Liste (Schwanz). Funktionen über Listen folgen genau diesem Aufbau: Der Basisfall behandelt die leere Liste direkt, der allgemeine Fall verarbeitet den Kopf und ruft sich rekursiv für den Schwanz auf. Die Länge einer leeren Liste ist 0, sonst 1 plus die Länge des Schwanzes – ganz einfach.

**Bäume** funktionieren genauso. Ein Binärbaum ist entweder ein leeres Blatt (Basisfall) oder ein Knoten mit einem Wert und zwei Teilbäumen (allgemeiner Fall). Die Tiefe eines Baumes berechnet man rekursiv: Man fragt die Tiefen des linken und rechten Teilbaums ab und nimmt das Maximum. Ein leeres Blatt hat Tiefe 0.

Der Vorteil: Wenn man den Aufbau der Datenstruktur kennt, ergibt sich die Struktur der passenden Funktion von selbst. Das macht rekursive Programme über solche Typen leichter zu schreiben und zu verstehen [7].

# 5 Rekursion in der funktionalen Programmierung

In Sprachen wie Java benutzt man `for`- oder `while`-Schleifen für Wiederholungen – dabei wird eine Variable bei jedem Durchlauf verändert. In rein funktionalen Sprachen hingegen bleiben Werte nach der Zuweisung unveränderlich, sogenannte **Immutability**. Schleifen, die auf veränderbaren Variablen basieren, passen dort nicht ins Konzept [1].

Deshalb übernimmt Rekursion dort die Rolle der Schleifen: Statt eine Variable hochzuzählen, übergibt man den neuen Wert als Argument an den nächsten Aufruf. Das Ergebnis ist dasselbe – nur ohne veränderbaren Zustand [8].

**Scala** ist ein gutes Beispiel, weil es beides unterstützt: die objektorientierte Welt von Java und funktionale Konzepte. Ein direkter Vergleich zeigt den Unterschied:

| **Java – iterativ** | **Scala – rekursiv** |
|---|---|
| `long factorial(int n) {`<br>`  long result = 1;`<br>`  for (int i = 1; i <= n; i++) {`<br>`  result = result * i;`<br>`  }`<br>`  return result;`<br>`}` | `def factorial(n: Int): Long =`<br>`  if n == 0 then 1`<br>`  else n * factorial(n - 1)` |

Tabelle 3 — Fakultät iterativ in Java vs. rekursiv in Scala

Beide Varianten liefern dasselbe Ergebnis. Die Java-Version verändert `result` und `i` mit jedem Schleifendurchlauf. Die Scala-Version kommt ohne veränderbare Variablen aus – der Ablauf ergibt sich allein durch die Struktur der rekursiven Aufrufe. In Scala kann man außerdem mit `@tailrec` sicherstellen, dass der Compiler endrekursive Funktionen automatisch in eine Schleife umwandelt – dann entsteht kein Stacküberlauf [5].

In Haskell, einer rein funktionalen Sprache, ist dieser Stil noch konsequenter. Funktionen werden durch **Pattern Matching** definiert: Man schreibt für jeden Fall eine eigene Zeile, und

Haskell wählt automatisch die passende. Das folgende Beispiel zeigt die Fakultät und eine Funktion zum Summieren einer Liste – beide ohne eine einzige veränderbare Variable:

```haskell
-- Fakultät: Basisfall und allgemeiner Fall
factorial :: Int -> Int
factorial 0 = 1                          -- Basisfall
factorial n = n * factorial (n - 1)      -- allgemeiner Fall

-- Summe einer Liste: leere Liste und Kopf/Schwanz
sumList :: [Int] -> Int
sumList []     = 0                       -- Basisfall: leere Liste
sumList (x:xs) = x + sumList xs          -- Kopf + Summe des Rests
```

Listing 2 — Fakultät und Listensumme rekursiv in Haskell

Die Struktur entspricht exakt dem, was wir bisher in Java und Scala gesehen haben – nur schreibt man in Haskell keine `if`-Bedingungen, sondern Gleichungen pro Fall. Das `(x:xs)` ist Haskell-Notation für „erstes Element `x` und restliche Liste `xs`„ und entspricht dem Kopf-Schwanz-Prinzip der rekursiven Listenstruktur [7].

Neben der direkten Rekursion bieten funktionale Sprachen höherwertige Funktionen wie `map`, `filter` oder `fold`, die häufige Rekursionsmuster kapseln. Die Rekursion steckt dabei weiterhin drin – sie ist nur versteckt und muss nicht jedes Mal neu geschrieben werden [7].

# 6 Fazit

Rekursion ist kein komplizierter Trick, sondern ein natürliches Denkprinzip: Man löst ein Problem, indem man es auf eine kleinere Version desselben Problems zurückführt. Das haben Menschen schon vor mehr als 2000 Jahren gemacht – mit dem euklidischen Algorithmus.

In der funktionalen Programmierung ist Rekursion besonders wichtig, weil es ohne veränderliche Variablen keine Schleifen geben kann. Wer versteht, wie induktive Datentypen aufgebaut sind, kann darüber fast automatisch korrekte rekursive Funktionen schreiben. Konzepte wie Endrekursion zeigen außerdem, dass rekursive Programme auch effizient sein können – nicht nur elegant.

# Literatur


[1] P. Pepper, *Funktionale Programmierung in OPAL, ML, Haskell und Gofer*. Springer Verlag, 2006.

[2] inf-schule.de, „Rekursive Funktionen – Konzept“. Zugegriffen: 16. März 2026. [Online]. Verfügbar unter: https://inf-schule.de/imperative-programmierung/python/konzepte/rekursion/konzept_rekursivefunktion

[3] M. Lenz, „Rekursion verstehen“. Zugegriffen: 16. März 2026. [Online]. Verfügbar unter: https://perlgeek.de/de/artikel/rekursion

[4] G. Lorenz, *Funktionale Modellierung und Rekursion: Praxis – Didaktik – Theorie*. München: Oldenbourg Verlag, 2012.

[5] M. Odersky, L. Spoon, und B. Venners, *Programming in Scala*, 5. Aufl. Artima Press, 2021.

[6] R. Sedgewick und K. Wayne, *Algorithmen und Datenstrukturen*. München: Pearson, 2014.

[7] M. Block und A. Neumann, *Haskell Intensivkurs*. Springer Verlag, 2011.

[8] entwickler.de, „Rekursion – die Eleganz der Wiederholung“. Zugegriffen: 16. März 2026. [Online]. Verfügbar unter: https://entwickler.de/machine-learning/rekursion-die-eleganz-der-wiederholung